

Universidad Técnica Federico Santa María

Departamento de Industrias

Asignatura: Sistemas de Información para la Gestión — ICN-292

Profesores: José Miguel González Paul — José Luis Sáez Tamayo



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Flujos en n8n para solicitudes de devolución

AndesHogar SpA

Laboratorio 3

Nombre: Alex Rebolledo

RUT: 21.500.500-3

Semilla “S” = 500

Repositorio: <https://github.com/rebolledocontrerasalex-alt/ICN292-Lab3-Rebolledo-Alex>

Resumen ejecutivo

El presente informe contiene la información del proceso realizado para construir un producto mínimo viable en n8n cuyo fin es automatizar el triage de solicitudes de devolución del canal de comercio en línea de AndesHogar SpA, reemplazando la revisión manual que toma en promedio tres días hábiles frente a las veinticuatro horas que promete la política publicada. El producto mínimo viable (MVP), recibe cada solicitud por webhook, aplicando las siete reglas de negocio en el orden fijado por la política, registrando el resultado junto con su motivo y respondiendo de inmediato al cliente. El código principal, más los tres flujos de trabajo y el informe en los formatos pedidos, se encuentran disponibles también en el repositorio de Github en el siguiente link: <https://github.com/rebolledocontrerasalex-alt/ICN292-Lab3-Rebolledo-Alex>.

El sistema se compone de tres flujos independientes. El primero (triage) implementa el árbol de decisión completo usando únicamente los nodos: Webhook, Switch, Edit Fields, HTTP Request, Merge, Google Sheets y Respond to Webhook, sin escribir ningún código. El segundo (emisor) simula el envío de las quince solicitudes de prueba contra el webhook de triage. El tercero (resumen) corre todos los días a las 20:00 y consolida en un solo mensaje cuántas solicitudes se procesaron por ruta, el monto total y la tasa de aprobación automática del día.

Al procesar las quince solicitudes asignadas, siete terminaron en RECHAZO, seis en REVISION y dos en APROBACION automática (ver tabla de resultados en la Parte A). Ningún caso activó de forma pura las reglas de estado del producto (producto dañado o con fallas), porque en los datos entregados esas solicitudes ya excedían el plazo o el monto, que se evalúan antes según el orden de precedencia de la política, un hallazgo que se explica en detalle más abajo y que evidencia que la regla de precedencia sí es determinante sobre el resultado final.

Durante la implementación surgió un problema real y no buscado: el servicio público mindicator.cl, usado para expresar el monto en UF, falló de forma intermitente al ser llamado desde el servidor de n8n, cortando la conexión sin previo aviso. Ese incidente se documenta en la Parte C como una de las tres entradas que hacen fallar un nodo, y llevó a rediseñar esa parte del flujo con reintentos, timeout acotado y una estrategia de On Error que permite seguir clasificando la solicitud, aunque el dato de UF no esté disponible ese día.

La recomendación final, desarrollada al cierre del informe, es que el MVP sí resuelve el cuello de botella identificado y conviene evolucionarlo al sistema definitivo, pero no todavía tal cual está: antes hay que agregar un Error Workflow (hoy un fallo del flujo programado pasaría inadvertido), incorporar una validación de SKU contra el catálogo (un vacío real detectado en la Parte C) y decidir si mindicator.cl es una dependencia aceptable en producción o si conviene un proveedor con garantías de disponibilidad.

Parámetros según semilla personal

Según el RUT de quien realizó este trabajo, la semilla correspondiente es de $S = 500$. Así, los parámetros para este laboratorio fueron:

Parámetro	Cálculo	Valor
S	Tres últimos dígitos del RUT	500
U (Umbral del monto)	$30000 + 1000(S \bmod 50) = 30000 + 1000*0$	\$30000

D (Plazo máximo)	$7 + 7*(S \bmod 4) = 7+7*0$	7 días
------------------	-----------------------------	--------

Tabla 1: Parámetros utilizados según semilla indicada

Parte A: Flujo de triage

Arquitectura del flujo de trabajo (workflow)

El workflow triage.json se compone de doce nodos, de los cuales todos son declarativos, es decir, no posee ningún nodo Code. Están conectados tal como se aprecia en la imagen:

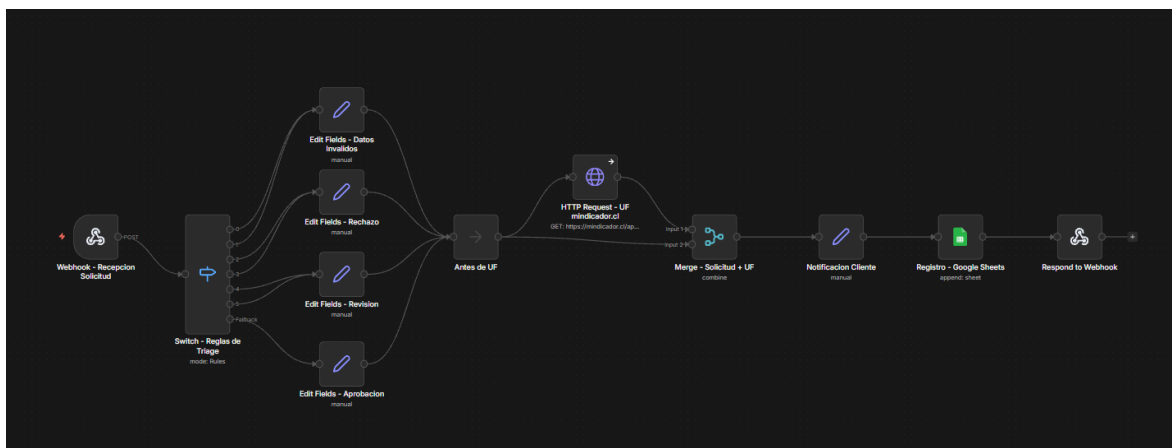


Figura 1: Flujo de trabajo de triage

1. Webhook (POST, path triage-devoluciones) recibe la solicitud en formato JSON, con todos los campos como texto tal como los enviaría un formulario web.
2. Switch en modo Rules evalúa, en el orden exacto de la política, las condiciones de datos inválidos, rechazo y revisión; el Fallback Output (sin condición) enruta a aprobación. Cada condición vive en su propia regla, con Convert Types (typeValidation loose) activado y Send data to all matching outputs desactivado, de modo que solo se dispara la primera regla que se cumple.

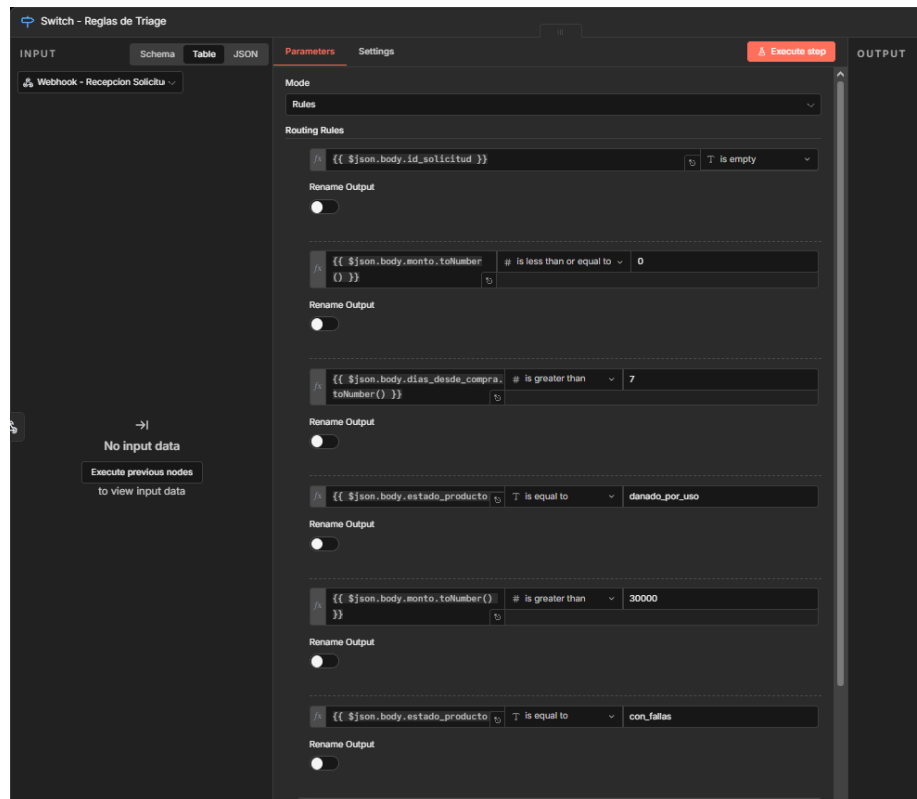


Figura 2: Configuraciones del nodo Switch

3. Cuatro nodos Edit Fields (uno por desenlace: Datos Inválidos, Rechazo, Revisión, Aprobación) escriben los campos ruta y motivo. Como dos causales distintas pueden llevar al mismo desenlace (por ejemplo, plazo vencido o estado dañado para Rechazo), el motivo se calcula con una expresión que distingue cuál de las dos condiciones fue la que realmente se cumplió, en vez de duplicar el nodo.

4. Un nodo HTTP Request consulta <https://mindicador.cl/api/uf> para expresar el monto en UF, consumiendo el campo `serie[0].valor` de la respuesta (el valor de la UF más reciente publicado). Este nodo tiene timeout de 4 segundos y On Error = Continue, para que una caída de esta API externa (que efectivamente ocurrió de forma reiterada durante las pruebas) no derribe el resto del flujo y la solicitud se siga registrando igual, solo que sin el dato de monto_uf. Inicialmente el nodo tenía además reintentos automáticos (3 intentos, 2 segundos de espera cada uno), pero se quitaron a propósito: como la falla de mindicador.cl resultó ser persistente y no transitoria en el servidor usado, reintentar solo alargaba de forma impredecible cada ejecución (hasta 30 segundos) y eso terminaba generando otro problema, descrito más abajo, al escribir en Google Sheets.

5. Un nodo Merge (modo Combine, Combine by Position) junta, sin escribir código, los datos de la solicitud ya clasificada con la respuesta de mindicador.cl, evitando que esta última sobrescriba los campos de la solicitud (un problema real que apareció durante el desarrollo y que se documenta en la Parte C).

6. Un nodo de notificación (Edit Fields) arma el mensaje que se enviaría al cliente según su ruta, y un nodo Google Sheets (operación Append) registra el identificador, SKU, monto, ruta, motivo, los valores de U y D aplicados, el monto en UF y un timestamp cumpliendo la regla de trazabilidad de la política. Se usa Append simple y no Append or Update porque cada solicitud se registra una única

vez y nunca se actualiza: usar Append or Update habría requerido que el nodo primero buscara la fila correspondiente antes de escribir, y ese paso de búsqueda fue precisamente la causa de una pérdida de filas bajo ejecuciones concurrentes que se detectó durante las pruebas (detallada más abajo).

7. Respond to Webhook cierra la ejecución devolviendo al emisor el identificador, la ruta, el motivo y el mensaje.

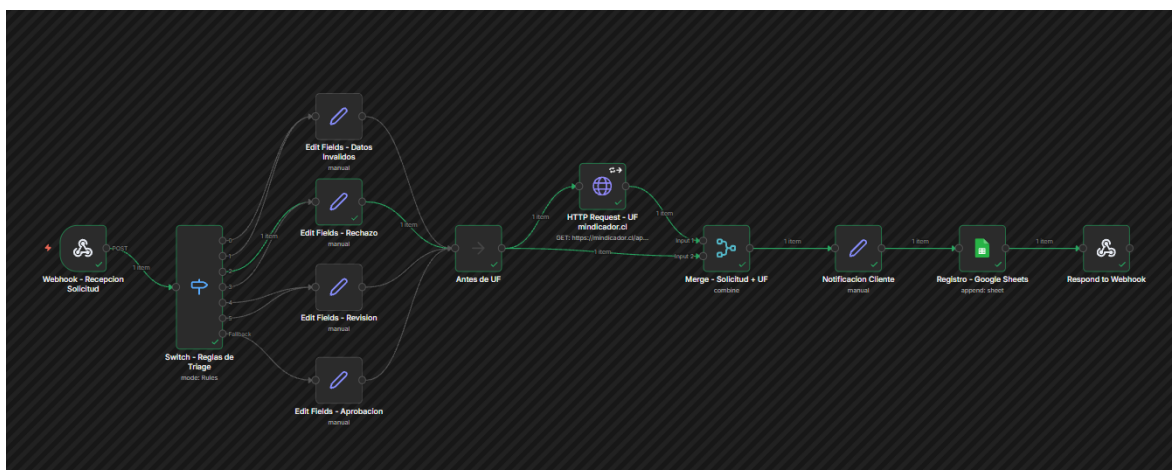


Figura 3: Ejecución exitosa del workflow de triage: los doce nodos corren en verde (sin errores), 1 ítem por nodo, hasta Respond to Webhook.

Resultado del procesamiento de las quince solicitudes

Se ejecutó el flujo emisor contra la Production URL del webhook de triage (con el flujo triage ya publicado), enviando las quince solicitudes provistas, incluidas las tres de control (DEV-C01, DEV-C02 y DEV-C03) y se registró el resultado de cada una en la hoja de Registro creada en Google Sheets:

id_solicitud	monto	días	estado_producto	ruta	motivo
DEV-1115	19990	3	sin_abrir	APROBACION	Cumple todos los requisitos
DEV-1116	69980	5	abierto_sin_uso	REVISION	Monto supera U=30000
DEV-1117	24990	12	con fallas	RECHAZO	Dias (12) supera D=7
DEV-1118	79990	4	sin_abrir	REVISION	Monto supera U=30000
DEV-1119	45990	25	danado_por_uso	RECHAZO	Dias (25) supera D=7
DEV-1120	89990	2	con fallas	REVISION	Monto supera U=30000
DEV-1121	289990	9	sin abrir	RECHAZO	Dias (9) supera D=7
DEV-1122	45990	2	sin_abrir	REVISION	Monto supera U=30000
DEV-1123	39980	15	danado_por_uso	RECHAZO	Dias (15) supera D=7
DEV-1124	49980	1	abierto_sin_uso	REVISION	Monto supera U=30000
DEV-1125	34990	22	sin abrir	RECHAZO	Dias (22) supera D=7

DEV-1126	79990	8	con fallas	RECHAZO	Dias (8) supera D=7
DEV-C01	9990	1	sin_abrir	APROBACION	Cumple todos los requisitos
DEV-C02	289990	2	sin_abrir	REVISION	Monto supera U=30000
DEV-C03	79990	29	abierto sin uso	RECHAZO	Dias (29) supera D=7

Tabla 2: Resultado en “Registro” con nodo enlazado a Google Sheets

De las quince, siete terminaron en RECHAZO, seis en REVISION y dos en APROBACION automática. Un detalle relevante para el informe es que ninguna solicitud cayó de forma pura en las reglas de estado del producto (RECHAZO_ESTADO por danado_por_uso, o REVISION_FALLA por con_fallas): en los dos casos con estado danado_por_uso (DEV-1119 y DEV-1123) y en los tres con estado con_fallas (DEV-1117, DEV-1120 y DEV-1126), el plazo o el monto ya excedían el límite antes de llegar a evaluarse el estado del producto, porque esas reglas se revisan primero según el orden de precedencia de la política. Esto demuestra en la práctica que el orden de las reglas, y no solo su contenido, determina el resultado final, que es justamente el punto que la regla 1 de precedencia buscaba probar.

Un segundo hallazgo real, no buscado, surgió al registrar el resultado en Google Sheets: aunque las quince ejecuciones del triage terminaron sin error, la hoja Registro quedó con menos de quince filas (14 de 15 en la corrida documentada abajo). La causa no está en el workflow: el nodo Google Sheets usa la operación Append, y la propia API de Google Sheets no asegura que dos peticiones de escritura muy cercanas en el tiempo no terminen pisándose entre sí al determinar la fila siguiente disponible. Se mitigó bastante el problema espaciando el envío de las solicitudes desde el emisor (batching de a un ítem cada 5 segundos) y quitando los reintentos del nodo de mindicator.cl (que antes alargaban de forma impredecible la duración de cada ejecución y desincronizaban el espaciado), pero no se eliminó por completo, por lo que es una limitación de escribir concurrentemente contra una API externa, no un defecto de diseño del flujo.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	id_solicitud	sku	monto	dias_desde_compra	estado_producto	ruta	motivo	U	D	monto_uf	email_cliente	timestamp	uf_error
2	DEV-1115	AU-210	19990	3	sin_abrir	APROBACION	cumple todos los	30000	7		cliente1@exam	2026-09-15T22:00	[object Object]
3	DEV-1116	TE-330	99980	5	abierto_sin_uso	REVISION	monto (99980) s	30000	7		cliente2@exam	2026-09-15T22:00	[object Object]
4	DEV-1119	PL-120	45990	25	danado_por_uso	RECHAZO	dias_desde_con	30000	7		cliente5@exam	2026-09-15T22:00	[object Object]
5	DEV-1118	MW-150	79990	4	sin_abrir	REVISION	monto (79990) s	30000	7		cliente4@exam	2026-09-15T22:00	[object Object]
6	DEV-1121	LV-330	289990	9	sin_abrir	RECHAZO	dias_desde_con	30000	7		cliente7@exam	2026-09-15T22:00	[object Object]
7	DEV-1120	AS-260	89990	2	con_fallas	REVISION	monto (89990) s	30000	7		cliente5@exam	2026-09-15T22:00	[object Object]
8	DEV-1123	AU-210	39980	15	danado_por_uso	RECHAZO	dias_desde_con	30000	7		cliente9@exam	2026-09-15T22:00	[object Object]
9	DEV-1122	PL-120	45990	2	sin_abrir	REVISION	monto (45990) s	30000	7		cliente8@exam	2026-09-15T22:00	[object Object]
10	DEV-1124	HV-050	49980	1	abierto_sin_uso	REVISION	monto (49980) s	30000	7		cliente10@exam	2026-09-15T22:00	[object Object]
11	DEV-1126	MW-150	79990	8	con_fallas	RECHAZO	dias_desde_con	30000	7		cliente12@exam	2026-09-15T22:00	[object Object]
12	DEV-1125	TE-330	34990	22	sin_abrir	RECHAZO	dias_desde_con	30000	7		cliente11@exam	2026-09-15T22:00	[object Object]
13	DEV-C01	CB-010	9990	1	sin_abrir	APROBACION	cumple todos los	30000	7		control01@exam	2026-09-15T22:00	[object Object]
14	DEV-C02	LV-330	289990	2	sin_abrir	REVISION	monto (289990) s	30000	7		control02@exam	2026-09-15T22:00	[object Object]
15	DEV-C03	MW-150	79990	29	abierto_sin_uso	RECHAZO	dias_desde_con	30000	7		control03@exam	2026-09-15T22:00	[object Object]

Figura 4: Hoja de Registro enlazada al nodo de Google Sheets, obteniendo 14 de 15 solicitudes en las filas escritas.

Parte B: Flujo de resumen

El tercer workflow (resumen.json) usa un Schedule Trigger configurado con la expresión cron 0 20 * * *, es decir, todos los días a las 20:00 hora de Santiago. Se eligió ese horario porque coincide con el cierre habitual de la jornada de atención al cliente: a esa hora ya se procesaron todas las solicitudes

que entraron durante el día, por lo que el resumen refleja el total real de la jornada en vez de un corte parcial.

El flujo lee todas las filas de la hoja Registro, las filtra para quedarse solo con las de la fecha de hoy, y usa el nodo Summarize para agrupar por ruta, calculando la cantidad de solicitudes y el monto total de cada una. Un nodo Code arma un único mensaje de texto con el desglose por ruta, el total de solicitudes, el monto total del día y la tasa de aprobación automática (aprobaciones sobre el total).

Para el caso de un día sin solicitudes, el nodo Filter se configuró con la opción Always Output Data activada, así, aunque no haya ninguna fila que coincida con la fecha de hoy, el flujo no se detiene en silencio, como ocurriría por defecto en n8n cuando un nodo no tiene ítems de salida, sino que sigue corriendo con un ítem vacío, y el nodo siguiente detecta esa condición y arma el mensaje No se registraron solicitudes hoy en vez de simplemente no enviar nada.

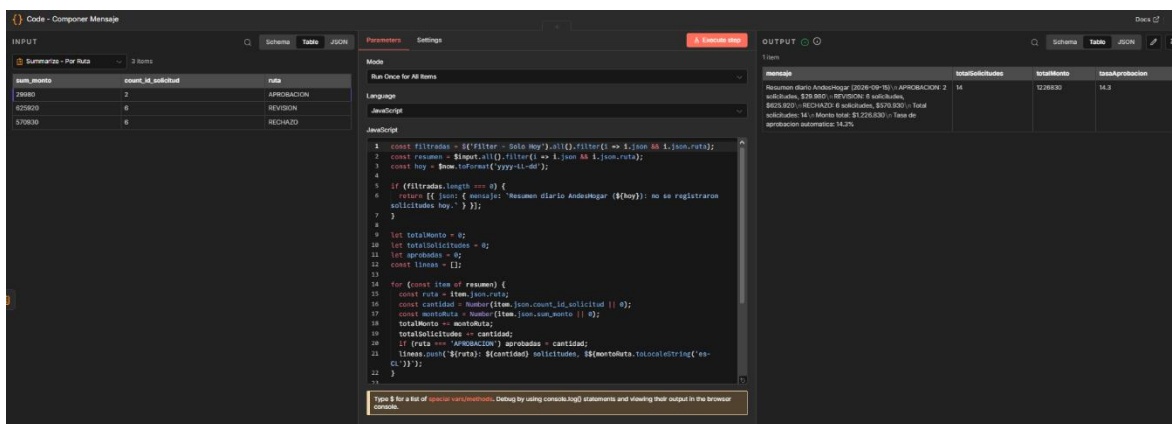


Figura 5: Ejecución del flujo de trabajo de resumen enfocado en el nodo Code

Parte C: Errores y casos borde

Tres entradas inválidas

Falla real del nodo HTTP Request (mindicador.cl):

Carga enviada: una solicitud válida cualquiera (por ejemplo, DEV-1115) contra el webhook de triage, en un momento en que el servidor de n8n no logró completar la conexión hacia mindicador.cl.

Comportamiento esperado: el nodo HTTP Request obtiene el valor de la UF vigente y el registro queda con monto_uf calculado.

Comportamiento observado: el nodo falló repetidamente con el error "The connection to the server was closed unexpectedly, perhaps it is offline", pese a que el mismo llamado funcionaba sin problema desde un navegador. Este fue un error real, no forzado, que hizo caer varias ejecuciones completas del triage hasta que se configuraron los reintentos, el timeout de 8 segundos y la estrategia On Error = Continue combinada con el nodo Merge para no perder los datos de la solicitud.

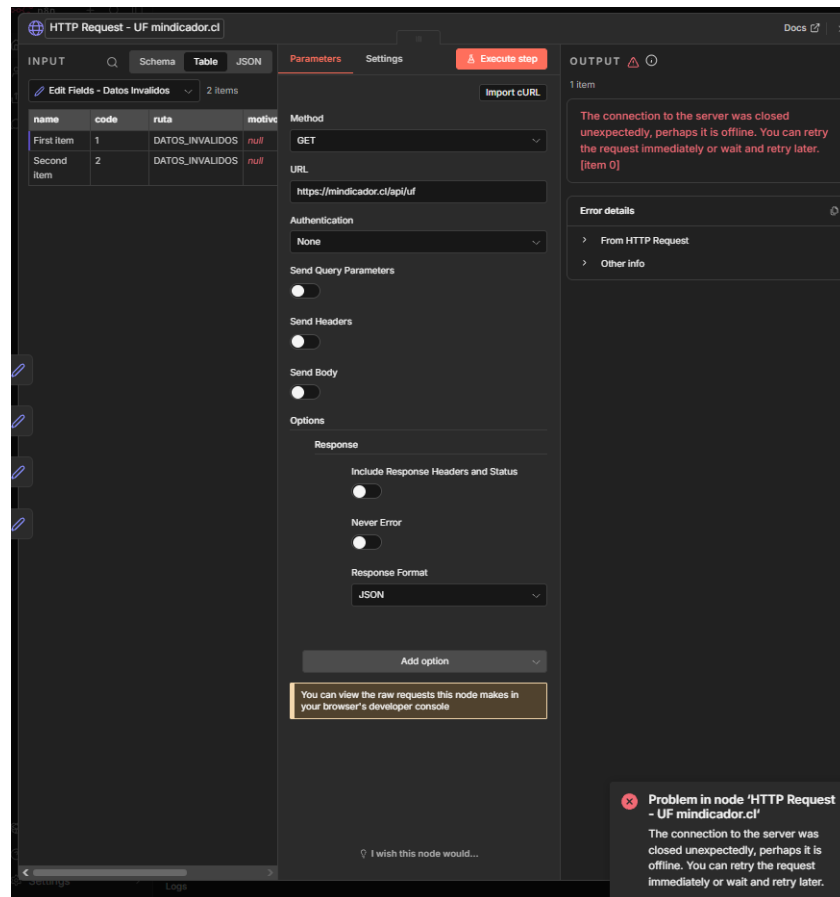


Figura 6: falla real del nodo HTTP Request

Campo obligatorio vacío:

Carga enviada: { "id_solicitud": "", "sku": "PL-120", "monto": "45990", "dias_desde_compra": "2", "estado_producto": "sin_abrir", "email_cliente": "cliente@example.cl" }.

Comportamiento esperado y observado: la regla 1 (String · is empty sobre id_solicitud) se cumple primero que cualquier otra, por lo que la solicitud se clasifica como DATOS_INVALIDOS con motivo "falta id_solicitud".

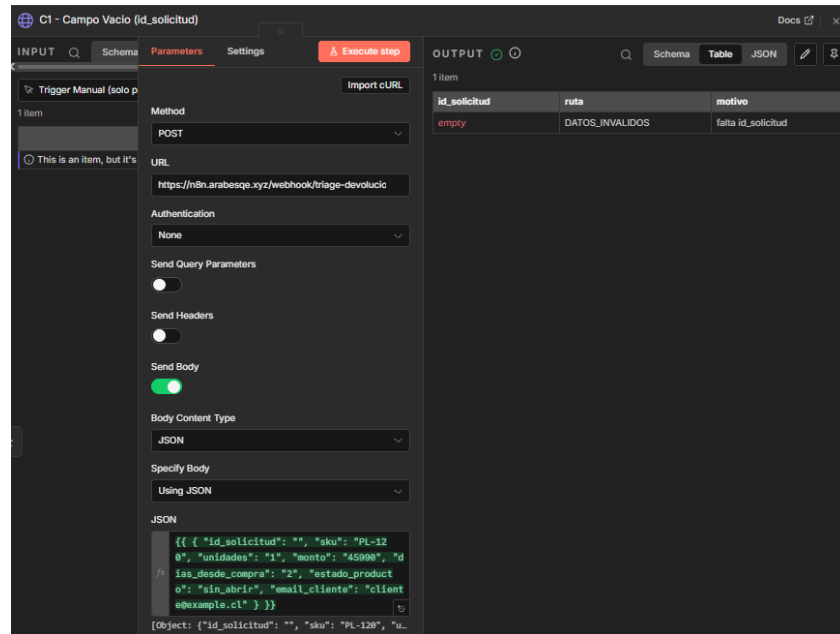


Figura 7: Solicitud con id_solicitud vacío

Monto no positivo:

Carga enviada: la misma solicitud anterior, pero con id_solicitud presente y "monto": "0".

Comportamiento esperado y observado: la regla 2 ($\text{Number} \cdot \text{lte } 0$ sobre $\text{monto.toNumber}()$) se cumple, clasificando la solicitud como DATOS_INVALIDOS con motivo "monto no es mayor a cero".

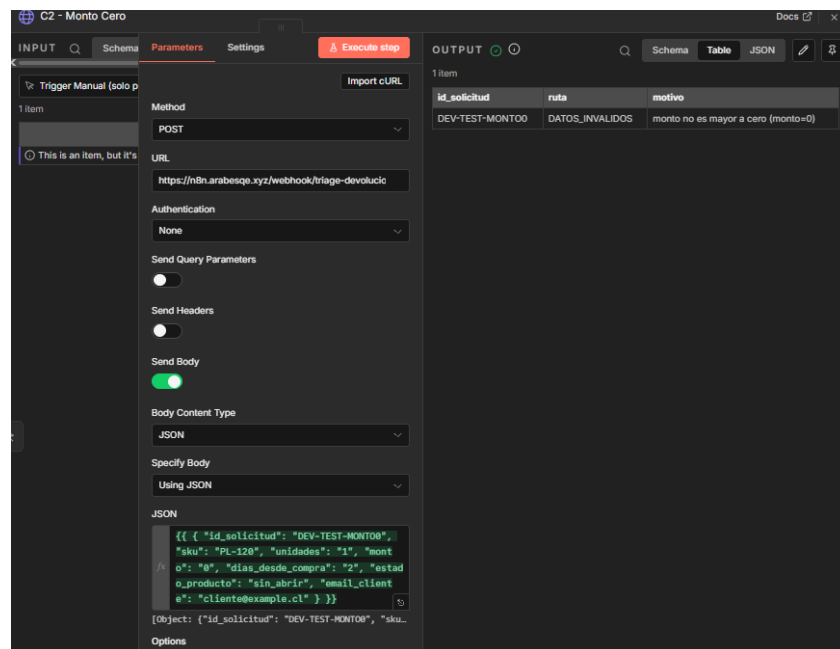


Figura 8: Solicitud con monto="0"

Los tres valores de On Error

n8n ofrece tres estrategias de manejo de errores por nodo: Stop Workflow Execution detiene toda la ejecución en el primer error, sin excepción; Continue permite que el flujo siga corriendo como si el nodo hubiese entregado su salida regular, aunque haya fallado; y Continue Using Error Output también deja seguir el flujo, pero enruta específicamente los items que fallaron hacia una segunda salida del nodo, para poder tratarlos de forma explícita (por ejemplo, registrarlos aparte).

En este workflow, el nodo HTTP Request - UF mindicator.cl quedó configurado en Continue, porque el dato que aporta (el monto expresado en UF) es complementario y no crítico para la decisión de negocio: si la API externa falla, igual conviene que la solicitud se siga clasificando y registrando, solo que sin ese dato adicional. El resto de los nodos (Switch, los Edit Fields, Google Sheets y Respond to Webhook) se dejaron en la configuración por defecto, Stop Workflow Execution, porque un fallo ahí sí compromete la integridad del registro: es preferible que la ejecución se detenga y quede visible en el historial de errores a que, por ejemplo, el nodo de registro falle silenciosamente y una solicitud real quede sin trazabilidad.

Qué es un Error Workflow

Un Error Workflow es un workflow especial que se asigna, desde la pestaña Settings de otro workflow, para que se ejecute automáticamente cada vez que ese workflow falla. Típicamente se usa para notificar el fallo (por correo, Slack, etc.), registrarlo en algún lugar centralizado o disparar un reintento. Ninguno de los tres workflows de este laboratorio tiene un Error Workflow configurado.

Esto importa especialmente para el workflow de resumen, porque corre solo, disparado por el Schedule Trigger, sin que nadie esté mirando la pantalla a las 20:00. Si ese flujo falla, por ejemplo, porque Google Sheets rechaza la credencial ese día, o porque el nodo Summarize se queda sin memoria con un volumen alto de datos sin un Error Workflow que avise, el fallo simplemente queda registrado en el historial de ejecuciones de n8n y nadie se entera hasta que alguien note que el resumen de ese día nunca llegó. Es un fallo silencioso, ya que el sistema no avisa que algo salió mal, solo deja de producir el resultado esperado.

Los límites

Las quince solicitudes entregadas no incluyen ningún caso de frontera exacta, así que se construyeron dos adicionales para comprobar que el flujo aplica correctamente la regla 6 (comparaciones inclusivas: monto = U se aprueba, días_desde_compra = D está en plazo):

id_solicitud	monto	días	estado_producto	ruta_esperada	motivo
DEV-LIM-01	30000	1	sin_abrir	APROBACION	monto = U (30000): no activa REVISION_MONTO porque la condición es estrictamente mayor a U
DEV-LIM-02	15000	7	sin_abrir	APROBACION	dias_desde_compra = D (7): no activa RECHAZO_PLAZO porque la condición es estrictamente mayor a D

Tabla 3: Límites encontrados

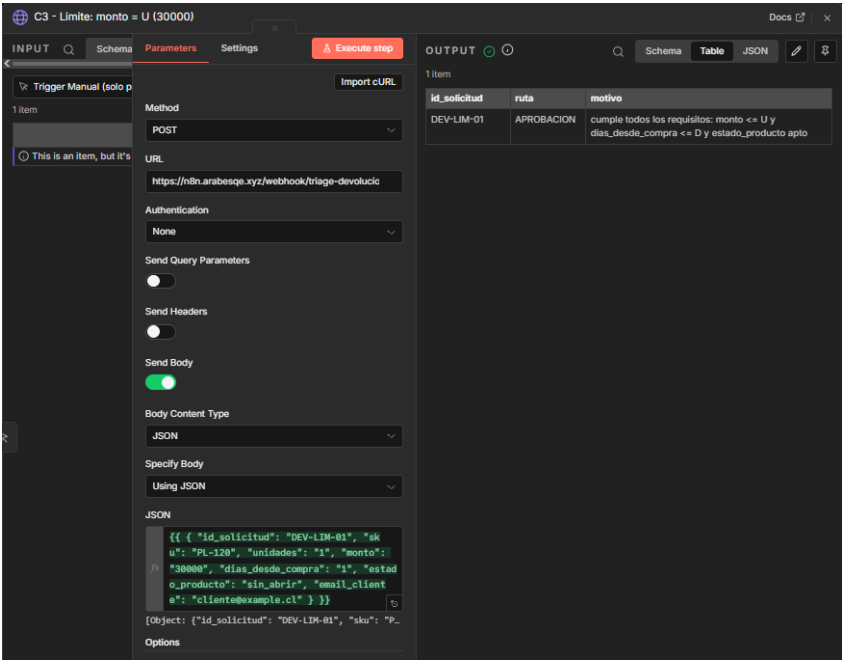


Figura 9: DEV-LIM-01 (monto = \$30.000): ruta APROBACION

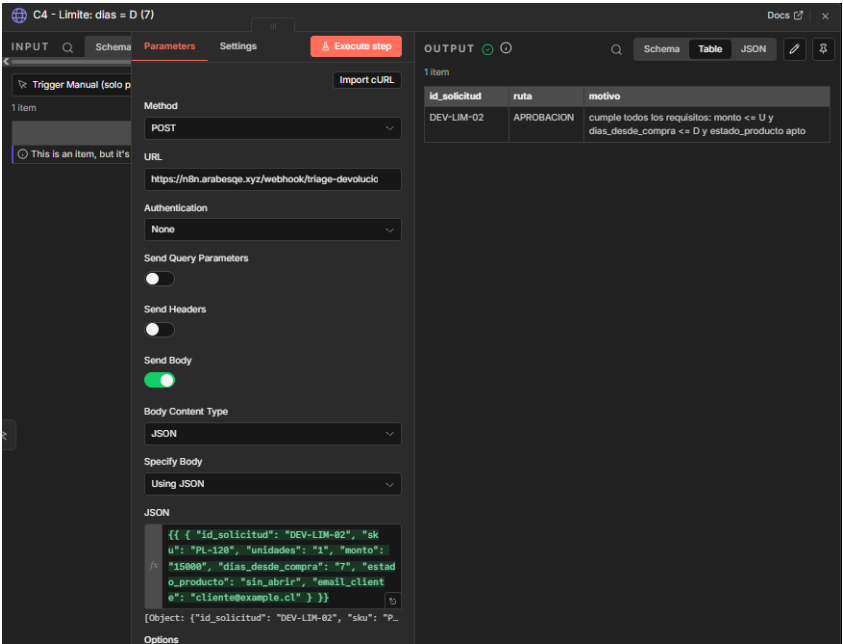


Figura 10: DEV-LIM-02 (días = 7 exacto): ruta APROBACION

El error de tipo a propósito

Se cambió temporalmente el tipo de la condición 2 del Switch (la que compara monto, originalmente Number · is equal to · 0) a String · is equal to · 0, y se envió DEV-C01 (\$9.990) para observar qué pasaba.

Comportamiento esperado: al no ajustar también el operador, se pensó que el nodo simplemente dejaría de detectar la condición (comparando '9990' contra '0' como texto) y la solicitud seguiría de largo hacia APROBACION sin ningún aviso.

Comportamiento observado: el Switch falló directamente, con el mensaje "Wrong type: '9990' is a number but was expecting a string [condition 0, item 0]". La causa es que la expresión del lado izquierdo, {{ \$json.body.monto.toNumber() }}, sigue forzando explícitamente un valor numérico sin importar qué tipo se le haya asignado a la condición; al declarar la condición como String, n8n exige que ese valor llegue como texto, y el propio Convert Types en modo loose no alcanza a resolver ese choque porque el tipo ya viene forzado desde la expresión, no desde el dato en sí.

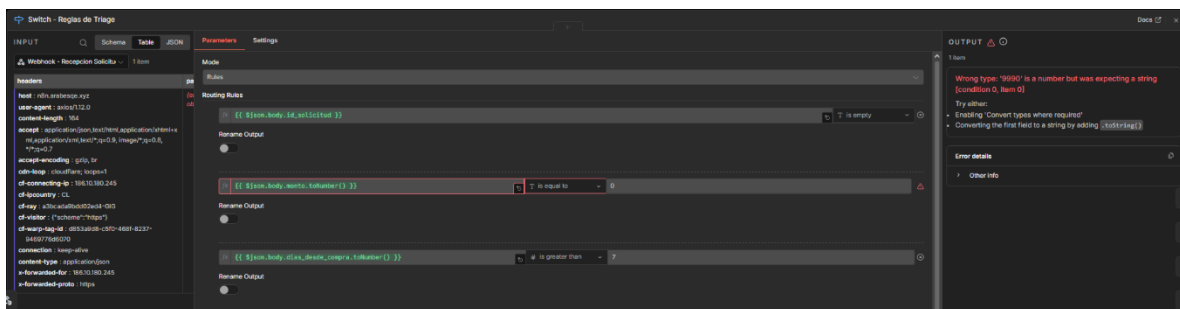


Figura 11: Error real del Switch al cambiar la condición 2 al tipo String

Esto es mejor evidencia que un simple resultado silencioso, porque muestra que el tipo de dato no es un detalle cosmético del nodo: es lo que define qué comparación se ejecuta y, en este caso, un descuadre entre el tipo forzado por la expresión y el tipo declarado en la condición basta para tumbar el nodo con un error explícito, sin necesidad de que el valor en sí esté mal formado. Después de la prueba se revirtió la condición a Number para dejar el flujo en su estado correcto.

Los dos casos que el flujo no puede resolver

Se probaron dos solicitudes adicionales que exponen límites reales del diseño actual:

Con "monto": "veinte mil" (texto no numérico), la expresión monto.toNumber() del Switch no puede convertir ese texto a un número válido, por lo que el nodo Switch falla en tiempo de ejecución en vez de clasificar la solicitud en alguna ruta. Esto ocurre porque la política asume que, aunque el monto llegue como texto, ese texto siempre representa un número; el flujo no valida ese supuesto antes de aplicar las reglas. Para detectarlo, se necesitaría una regla adicional, antes de las siete actuales, que verifique explícitamente si el campo monto es convertible a número, y en caso contrario clasifique la solicitud en una ruta de error de formato en lugar de dejar que el nodo falle.

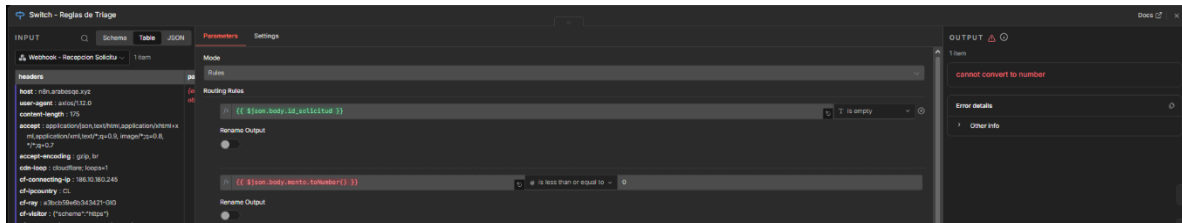


Figura 12: Falla real del Switch con monto="veinte mil"

Con "sku": "ZZ-999" (un SKU que no existe en el catálogo), la solicitud pasa todas las validaciones sin problema y se clasifica como si fuera perfectamente válida, porque ninguna de las siete reglas de la política revisa si el SKU corresponde a un producto real. El flujo confía en que el campo sku siempre viene bien formado, pero no lo contrasta contra ninguna fuente de verdad. Para detectarlo, se necesitaría agregar un paso que busque el sku recibido dentro de la hoja catálogo (por ejemplo, un nodo Google Sheets en modo Lookup, o una regla adicional en el Switch) y, si no aparece, enrute la solicitud a una ruta de SKU no encontrado en vez de dejarla avanzar como aprobación o revisión normal.

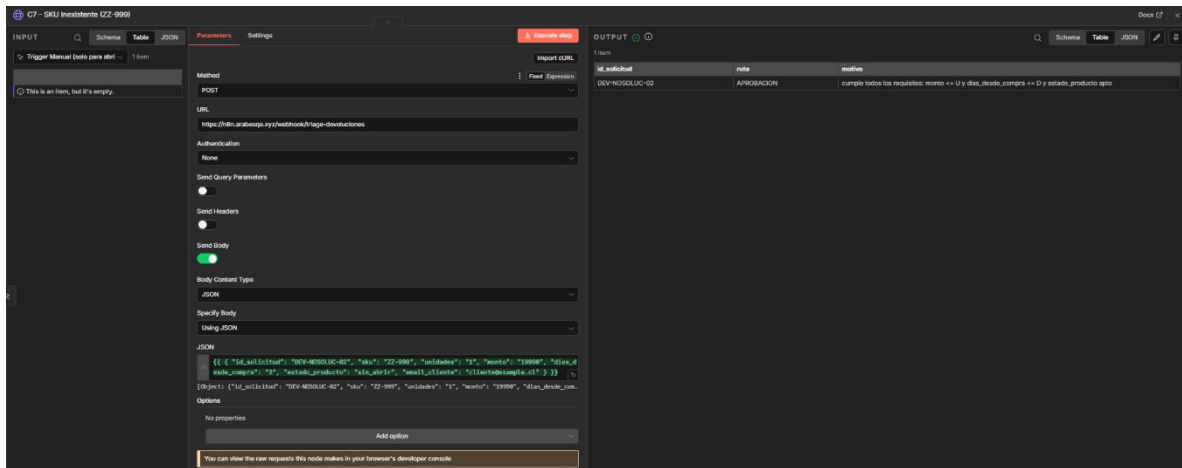


Figura 13: DEV-NOSOLUC-02 con sku="ZZ-999" (inexistente en el catálogo)

Bonus: La versión con código

Durante el desarrollo del laboratorio se probó, como parte del proceso de depuración, reemplazar el conjunto Switch + Edit Fields por un único nodo Code con las mismas siete reglas escritas como una cadena de condicionales if y else. La comparación entre ambas implementaciones, con la experiencia de haber construido y corregido las dos:

El nodo Code es más compacto: las siete reglas caben en un puñado de líneas, y agregar o reordenar una condición es una edición de texto rápida, sin tener que reconectar salidas del Switch ni duplicar Edit Fields. A cambio, es más difícil de auditar para alguien que no programa (un profesor, un encargado de Servicio al Cliente revisando la política) porque hay que leer código para entender qué hace, y un error de sintaxis o una condición mal puesta no se nota a simple vista como sí se nota un Switch con las reglas ordenadas visualmente una debajo de otra. El Switch + Edit Fields, en cambio, es más lento de modificar (cada regla nueva implica una condición en el Switch, un Edit Fields o una rama adicional, y las conexiones correspondientes), pero cualquiera puede seguir el flujo de decisiones mirando el canvas, sin necesidad de leer una sola línea de código, algo directamente relevante para este laboratorio, que pide explícitamente una versión sin código.

Si la política de AndesHogar cambiara todos los meses, como plantea la pregunta, elegiría el nodo Code, porque el costo de mantenerlo actualizado es menor y los cambios de reglas de negocio suelen ser más frecuentes que la necesidad de que alguien no técnico audite el flujo directamente en n8n. Aun así, mantendría en paralelo una documentación clara de las reglas vigentes (como la tabla de la Parte A de este informe) para no perder la trazabilidad que el Switch ofrece de forma nativa.

Conclusión y recomendación

El MVP sí resuelve el cuello de botella identificado en el contexto del laboratorio: automatiza exactamente la decisión que hoy alguien del equipo de Servicio al Cliente toma a mano, revisando cada solicitud una por una, y lo hace aplicando la política de forma consistente y trazable, con motivo registrado para cada caso. En ese sentido, conviene que evolucione hacia el sistema definitivo.

No se debería dejar pasar a producción tal cual está. Primero, porque no tiene un Error Workflow configurado, y un flujo que corre sin supervisión humana directa, como el resumen diario, necesita avisar cuando algo sale mal, no solo dejar de funcionar en silencio. Segundo, porque la Parte C mostró un vacío real en la validación, el flujo no verifica que el SKU exista en el catálogo, lo que en producción significaría aprobar o poner en revisión devoluciones de productos que ni siquiera vende la empresa. Tercero, porque la dependencia de mindicador.cl, una API pública, gratuita y sin garantías de disponibilidad, resultó ser menos confiable de lo esperado durante las pruebas; si el dato de UF se vuelve relevante para alguna decisión de negocio, convendría evaluar un proveedor. Y cuarto, porque el nodo de notificación al cliente hoy solo simula el envío del mensaje; antes de producción habría que conectarlo a un canal real de correo.

En conclusión, la arquitectura y la lógica de negocio del MVP están listas, pero antes de convertirlo en el sistema definitivo se necesita cerrar esos cuatro puntos, Error Workflow, validación de catálogo, decisión sobre la dependencia externa y notificación real, para que el sistema sea tan confiable como la política que automatiza.